Εφαρμογή ΠΙΝακοθήκης – Ομάδα 13 – Αναφορά εργασίας

Βάσεις Δεδομένων, 2025-2026

ΣΠΥΡΙΔΩΝ ΜΠΑΝΤΗΣ, Α.Μ. 1080556

ΚΩΝΣΤΑΝΤΙΝΟΣ ΧΑΛΑΡΗΣ, Α.Μ. 1097327

1 ΠΕΡΙΛΗΨΗ

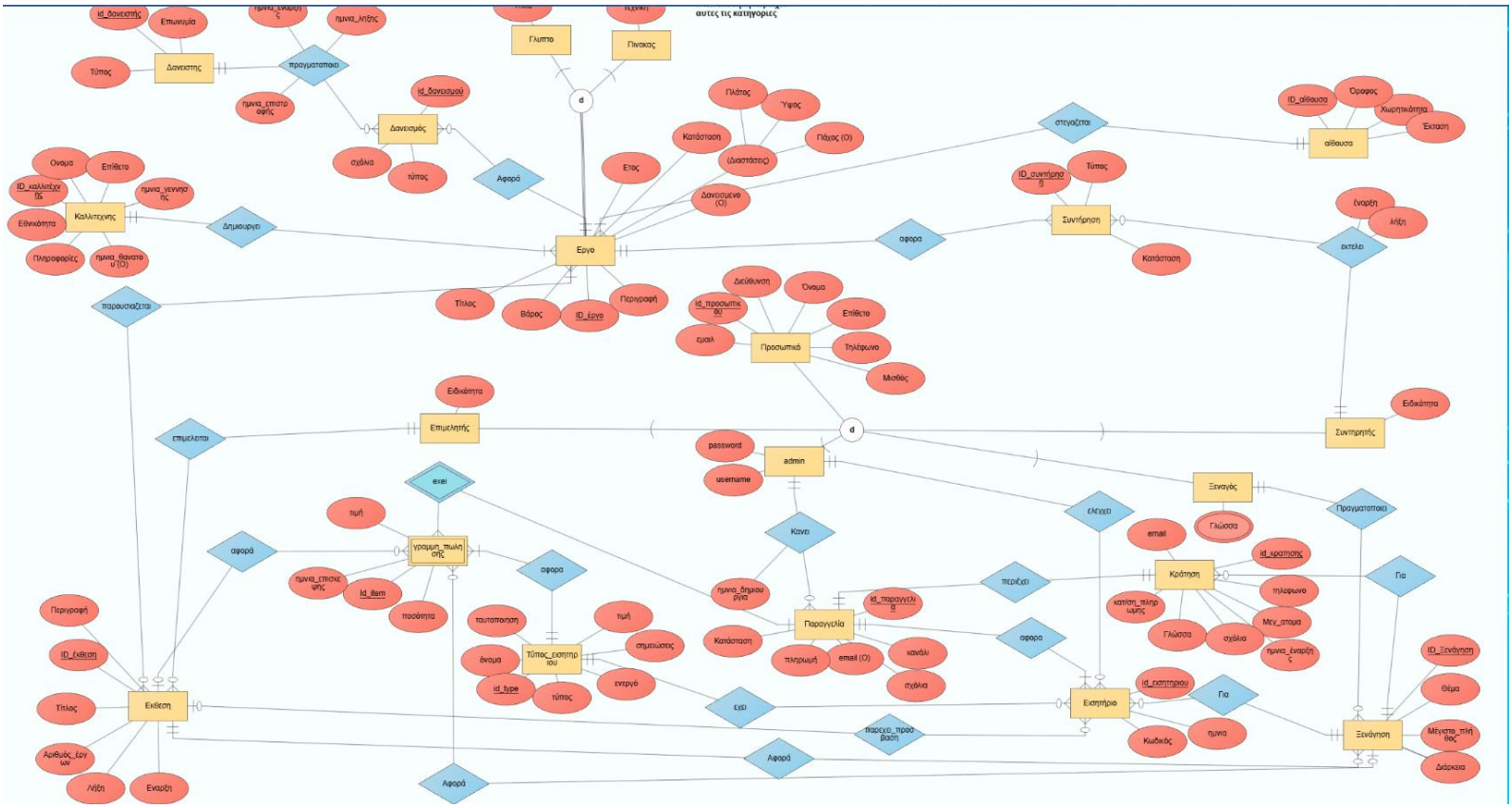
Η εργασία μας είχε ως στόχο τη σχεδίαση και υλοποίηση μιας βάσης δεδομένων για τη λειτουργία μιας πινακοθήκης και την ανάπτυξη μιας εφαρμογής διαχείρισης (admin dashboard) σε Python. Αρχικά, στη Φάση Α περιγράψαμε τον μικρόκοσμο της πινακοθήκης και τις παραδοχές χρήσης, σχεδιάζοντας εννοιολογικό μοντέλο (ERD) που καλύπτει έργα τέχνης, καλλιτέχνες, εκθέσεις, αίθουσες, προσωπικό με ρόλους (ξεναγοί/συντηρητές/επιμελητές/admins), συντηρήσεις, δανεισμούς, ξεναγήσεις/κρατήσεις και εισιτήρια. Στη συνέχεια μετασχηματίσαμε το μοντέλο σε λογικό σχεσιακό σχήμα με πίνακες, πρωτεύοντα και ξένα κλειδιά, καθώς και πίνακες συσχέτισης για σχέσεις M:N (π.χ. εκθέσεις–έργα). Στη Φάση Β υλοποιήσαμε τη βάση σε SQLite με εντολές δημιουργίας (DDL) δίνοντας έμφαση στην αναφορική ακεραιότητα και σε περιορισμούς/μοναδικότητες, όπως ο κανόνας ότι ένα έργο μπορεί να έχει μόνο έναν ενεργό δανεισμό τη φορά, μοναδικούς κωδικούς εισιτηρίων και καταγραφή ελέγχων εισιτηρίων σε scan log. Παράλληλα υλοποιήσαμε CRUD λειτουργίες και τυπικά SQL ερωτήματα για αναζητήσεις (λίστες έργων με φίλτρα, εκθέσεις ανά ημερομηνία, ενεργοί δανεισμοί, ιστορικό συντηρήσεων, πρόγραμμα ξεναγήσεων, στατιστικά εισιτηρίων κ.ά.). Τέλος, στη Φάση Γ αναπτύξαμε desktop εφαρμογή admin dashboard (PySide6) με μενού ενοτήτων (Έργα, Εκθέσεις, Συντήρηση, Προσωπικό, Δανεισμοί, Ξεναγήσεις, Εισιτήρια), αρχική οθόνη με συνοπτικά στοιχεία, στατιστικά και ημερολόγιο, και φόρμες εισαγωγής/επεξεργασίας δεδομένων. Το ενδιαφέρον της υλοποίησής μας είναι ότι συνδυάζει σωστό σχεσιακό σχεδιασμό με ρεαλιστικούς επιχειρησιακούς κανόνες (έλεγχος δανεισμών, αποφυγή επικαλυπτόμενων ξεναγήσεων, έλεγχος εγκυρότητας εισιτηρίων) και ένα χρηστικό περιβάλλον διαχείρισης που προσεγγίζει πραγματική εφαρμογή πινακοθήκης.

2 ΜΕΘΟΔΟΛΟΓΙΑ

Για την υλοποίηση της εργασίας ακολουθήσαμε μια σταδιακή και οργανωμένη διαδικασία, με στόχο το αποτέλεσμα να είναι σωστό ως προς τις αρχές των βάσεων δεδομένων, αλλά και πρακτικό ως προς τη χρήση του. Στο πρώτο στάδιο καθορίσαμε τις λειτουργικές απαιτήσεις του συστήματος, δηλαδή ποιες διαδικασίες μιας πινακοθήκης θέλουμε να υποστηρίζει η βάση δεδομένων και η εφαρμογή. Ξεκινήσαμε από τα βασικά σενάρια χρήσης: καταχώριση έργων και καλλιτεχνών, δημιουργία και διαχείριση εκθέσεων, διαχείριση αιθουσών και τοποθετήσεων/μετακινήσεων έργων, καταγραφή συντηρήσεων, δανεισμοί έργων σε φορείς, προγραμματισμός ξεναγήσεων με κρατήσεις, καθώς και σύστημα εισιτηρίων (έκδοση, πωλήσεις, έλεγχος στην είσοδο και καταγραφή ελέγχων). Με βάση αυτά αποφασίσαμε ποια δεδομένα είναι απαραίτητα να αποθηκεύονται και ποιοι κανόνες πρέπει να τηρούνται, ώστε το σύστημα να λειτουργεί ρεαλιστικά και να αποφεύγονται λάθη στα δεδομένα.

Στη συνέχεια προχωρήσαμε στον εννοιολογικό σχεδιασμό (EER/ERD). Αρχικά ορίσαμε τις κύριες οντότητες και τις σχέσεις τους και έπειτα συμπληρώσαμε τα γνωρίσματα που χρειάζονται για να υλοποιούνται τα παραπάνω σενάρια. Δώσαμε ιδιαίτερη προσοχή στις πληθικότητες, γιατί αυτές καθορίζουν τη σωστή υλοποίηση του σχεσιακού μοντέλου. Για παράδειγμα, μια έκθεση περιλαμβάνει πολλά έργα και ένα έργο μπορεί να συμμετέχει σε περισσότερες από μία εκθέσεις, άρα η σχέση είναι πολλών-προς-πολλούς. Αντίστοιχα, λειτουργίες όπως οι συντηρήσεις, οι δανεισμοί και οι έλεγχοι εισιτηρίων αποτελούν γεγονότα που επαναλαμβάνονται στο χρόνο, επομένως σχεδιάστηκαν με τρόπο που να επιτρέπουν την αποθήκευση ιστορικού (πολλαπλές εγγραφές ανά έργο ή εισιτήριο). Επίσης, το προσωπικό μοντελοποιήθηκε έτσι ώστε να υποστηρίζει διαφορετικούς ρόλους (ξεναγοί, συντηρητές, επιμελητές, διαχειριστές),

διατηρώντας κοινά στοιχεία εκεί που χρειάζεται.
Το τελικό εννοιολογικό μοντέλο της βάσης παρουσιάζεται παρακάτω (Εικόνα 1).



Εικόνα 1: Εννοιολογικό Μοντέλο - ERD

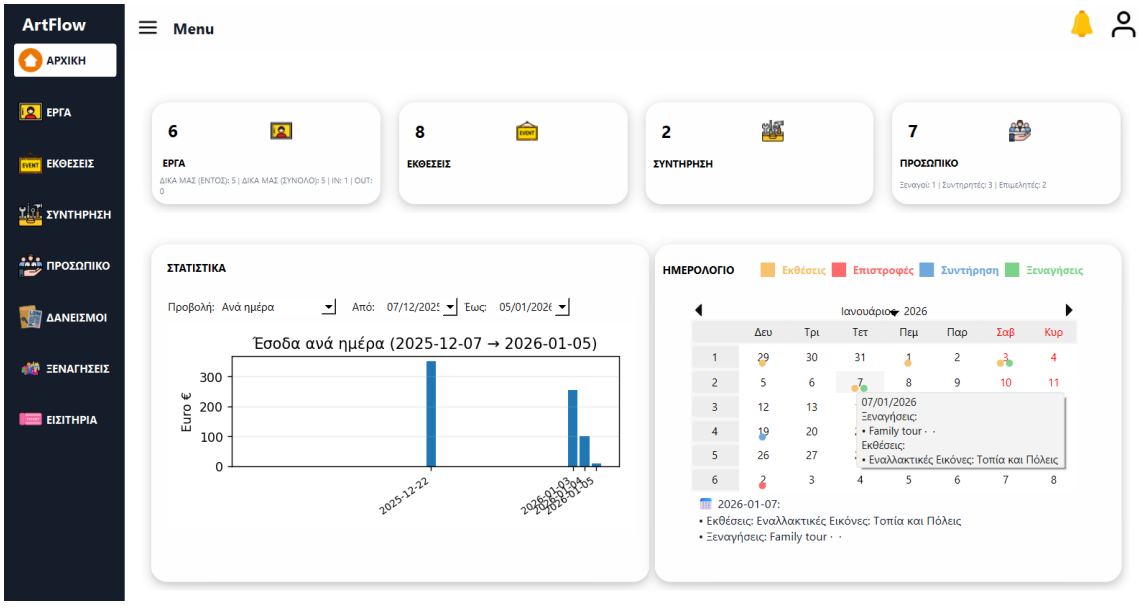
Αφού ολοκληρώθηκε το εννοιολογικό μοντέλο, το μετασχηματίσαμε σε λογικό σχεσιακό σχήμα εφαρμόζοντας τους κανόνες μετατροπής που διδαχθήκαμε. Οι σχέσεις πολλών-προς-πολλούς υλοποιήθηκαν με ενδιάμεσους πίνακες (π.χ. σύνδεση εκθέσεων–έργων), ενώ για δεδομένα που αφορούν γεγονότα ή ιστορικό δημιουργήθηκαν αντίστοιχοι πίνακες (π.χ. συντηρήσεις, δανεισμοί, κρατήσεις ξεναγήσεων, καταγραφές ελέγχων εισιτηρίων). Σε αυτό το στάδιο προσπαθήσαμε να κρατήσουμε τη δομή όσο γίνεται καθαρή, αλλά και αρκετά αναλυτική ώστε να υποστηρίζει τις λειτουργίες της εφαρμογής.Η δομή του λογικού σχεσιακού σχήματος φαίνεται παρακάτω (Εικόνα 2).

Εικόνα 2: Λογικό σχεσιακό μοντέλο της βάσης μας

Στη συνέχεια υλοποιήσαμε τη βάση σε SQLite, δημιουργώντας τους πίνακες με σαφή ορισμό πρωτευόντων κλειδιών, ξένων κλειδιών και περιορισμών ακεραιότητας. Δώσαμε έμφαση στο να αποφεύγονται ασυνέπειες στα δεδομένα και να εφαρμόζονται βασικοί επιχειρησιακοί κανόνες. Χαρακτηριστικά, εξασφαλίσαμε ότι ένα έργο δεν μπορεί να έχει πάνω από έναν ενεργό δανεισμό ταυτόχρονα, υλοποιήσαμε ελέγχους ώστε ο ίδιος ξεναγός να μην προγραμματίζεται σε επικαλυπτόμενα χρονικά διαστήματα και στο σύστημα εισιτηρίων χρησιμοποιήσαμε μοναδικούς κωδικούς και καταστάσεις (valid/used/canceled), ενώ καταγράψαμε κάθε έλεγχο εισόδου σε scan log για πλήρη ιχνηλασιμότητα.

Αφού σταθεροποιήθηκε το σχήμα, υλοποιήσαμε το επίπεδο πρόσβασης στη βάση (db layer), συγκεντρώνοντας τις βασικές λειτουργίες CRUD και τα ενδεικτικά ερωτήματα που χρειάζεται η εφαρμογή. Πάνω σε αυτό αναπτύξαμε την εφαρμογή σε Python με PySide6, ως admin dashboard, οργανωμένη σε ενότητες (έργα, εκθέσεις, συντήρηση, προσωπικό, δανεισμοί, ξεναγήσεις, εισιτήρια) και με φόρμες/διαλόγους για εισαγωγή και επεξεργασία δεδομένων.

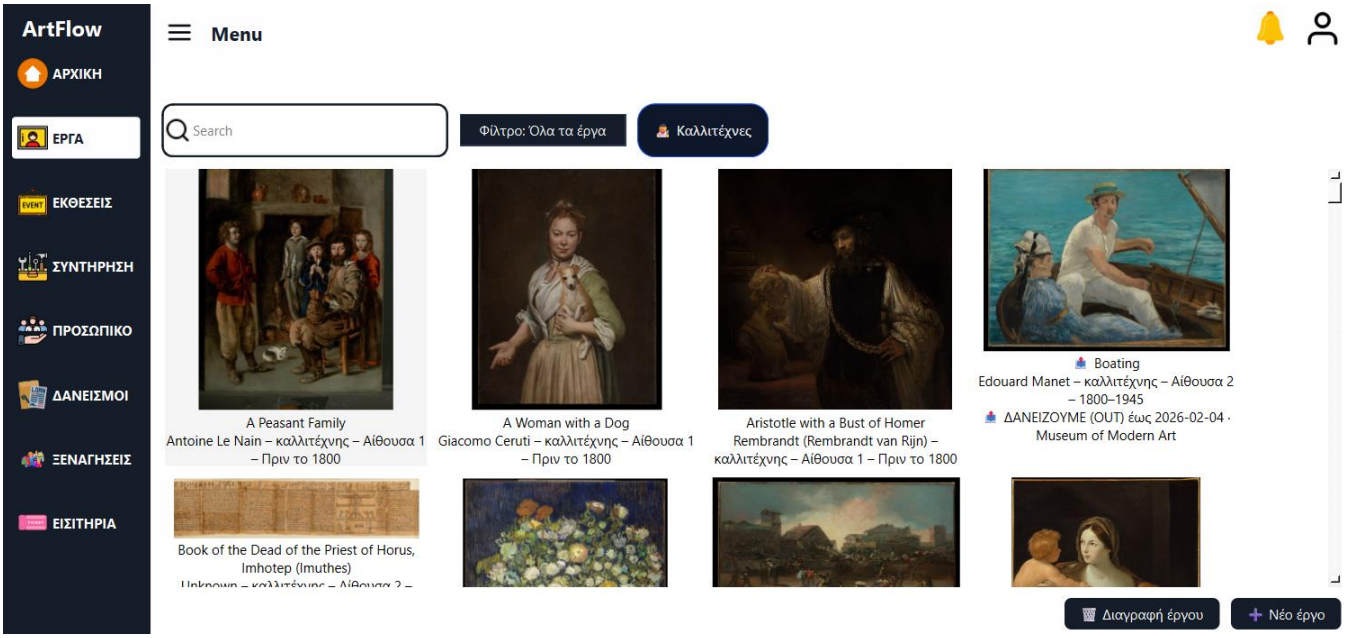
Στο επίπεδο της διεπαφής (UI) ξεκινήσαμε από τη σελίδα **Αρχική**, η οποία σχεδιάστηκε ως dashboard ώστε ο διαχειριστής να αποκτά άμεσα συνολική εικόνα της κατάστασης του συστήματος. Αρχικά υλοποιήσαμε τη δομή της σελίδας στο Qt Designer (cards, περιοχές στατιστικών και ημερολογίου) και στη συνέχεια τη συνδέσαμε με τη βάση μέσω του db layer, ώστε τα στοιχεία να ενημερώνονται δυναμικά από πραγματικά δεδομένα. Στο πάνω μέρος της Αρχικής δημιουργήσαμε cards με βασικούς μετρητές, όπως ανοιχτές συντηρήσεις, σύνολο προσωπικού (με διάκριση ανά ρόλο) και συνοπτική κατάσταση έργων (π.χ. συνολικά και διαχωρισμός ανά in/out δανεισμούς), τα οποία ανανεώνονται με συγκεντρωτικές συναρτήσεις “refresh”, ώστε να μπορεί η εφαρμογή να εμφανίζει γρήγορα την τρέχουσα εικόνα χωρίς ο χρήστης να μπαίνει σε επιμέρους ενότητες.Για την ενότητα Στατιστικά ενσωματώσαμε γράφημα με Matplotlib μέσα στο PySide6, με φίλτρα επιλογής χρονικού διαστήματος και τρόπου προβολής (ανά ημέρα/μήνα/τύπο εισιτηρίου). Για να είναι ομαλή η εμπειρία χρήσης και να αποφεύγονται συνεχόμενα redraws, εφαρμόσαμε μηχανισμό debounce με timer, ενώ προσθέσαμε ελέγχους εγκυρότητας (π.χ. αντιστροφή ημερομηνιών) και χειρισμό περιπτώσεων όπου δεν υπάρχουν δεδομένα, ώστε το γράφημα να εμφανίζει κατάλληλο μήνυμα αντί να αποτυγχάνει.Στην ενότητα Ημερολόγιο υλοποιήσαμε προσαρμοσμένο calendar, επειδή θέλαμε να φαίνονται ταυτόχρονα πολλαπλές κατηγορίες γεγονότων στην ίδια ημερομηνία. Για αυτό χρησιμοποιήσαμε χρωματική σήμανση με πολλαπλά “markers” ανά μέρα (π.χ. εκθέσεις, επιστροφές δανεισμών, συντηρήσεις, ξεναγήσεις), καθώς και φίλτρα με checkboxes ώστε ο χρήστης να επιλέγει τι θα προβάλλεται. Επιπλέον, προσθέσαμε προβολή λεπτομερειών τόσο με επιλογή ημερομηνίας (εμφάνιση λίστας γεγονότων) όσο και με hover tooltips, για γρήγορη ενημέρωση χωρίς περιττά clicks. Τέλος, συμπληρωματικά υλοποιήσαμε μηχανισμό ειδοποιήσεων (π.χ. επόμενες 7 ημέρες) ώστε ο admin να ενημερώνεται άμεσα για επερχόμενα γεγονότα.



Εικόνα 3: Αρχική σελίδα της εφαρμογής.

Στην ενότητα **Έργα** υλοποιήσαμε μια σελίδα διαχείρισης που λειτουργεί ως "κεντρικό σημείο" για την καταχώρηση και παρακολούθηση των αντικειμένων της συλλογής. Αντί για απλό πίνακα, επιλέξαμε μορφή gallery λίστας (QListWidget) με εικόνα/thumbnail και περιγραφικό κείμενο ανά έργο, ώστε ο admin να αναγνωρίζει γρήγορα τα έργα οπτικά. Η λίστα γεμίζει δυναμικά από το db layer (συνάρτηση που επιστρέφει έργα μαζί με στοιχεία καλλιτέχνη, αίθουσας και τυχόν ενεργό δανεισμό) και υποστηρίζει αναζήτηση (σε τίτλο και καλλιτέχνη) καθώς και φίλτρα (κατηγορία έργου, αίθουσα και κατάσταση δανεισμού). Επιπλέον, για καλύτερη εποπτεία εμφανίζουμε "σήμανση" στην πρώτη γραμμή του έργου (π.χ. ένδειξη αν είναι δανεισμένο IN/OUT), ενώ όταν υπάρχει ενεργός δανεισμός εμφανίζεται και συνοπτικό badge με πληροφορίες όπως προθεσμία και φορέας, ώστε να φαίνονται άμεσα τα κρίσιμα σημεία χωρίς να ανοίγει ο χρήστης ξεχωριστή καρτέλα.

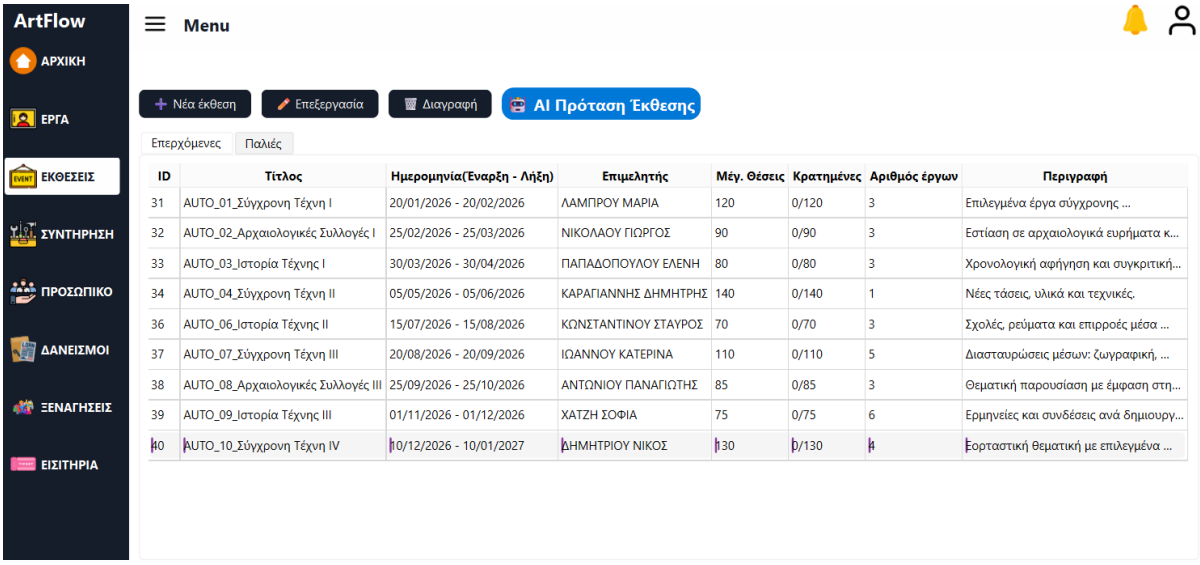
Για τις βασικές λειτουργίες διαχείρισης υλοποιήσαμε διαλόγους εισαγωγής/επεξεργασίας (WorkDialog), όπου γίνεται έλεγχος εγκυρότητας (π.χ. υποχρεωτικός τίτλος, σωστές τιμές αριθμητικών πεδίων), επιλογή κατηγορίας (πίνακας/γλυπτό με αντίστοιχα πεδία), επιλογή αίθουσας, επιλογή καλλιτέχνη από λίστα, καθώς και δυνατότητα εισαγωγής εικόνας. Η αποθήκευση γίνεται μέσω του db layer με ξεκάθαρες CRUD συναρτήσεις (INSERT/UPDATE), ενώ για διαγραφή εφαρμόζεται ασφαλής ροή με επιβεβαίωση και καθαρισμό εξαρτώμενων εγγραφών (π.χ. συντηρήσεις) ώστε να μην παραβιάζεται η ακεραιότητα. Τέλος, ενσωματώσαμε μέσα στο ίδιο dialog και τη διαχείριση δανεισμού του έργου, ώστε να εφαρμόζεται στην πράξη ο κανόνας "ένα έργο δεν μπορεί να έχει πάνω από έναν ενεργό δανεισμό": το σύστημα ελέγχει αν υπάρχει ενεργός δανεισμός και εμφανίζει ανάλογα την κατάσταση/κουμπιά (προσθήκη ή επιστροφή). Για έργα που είναι "εξωτερικά" (δανεισμένα προς εμάς) κλειδώνουμε και τη λογική κατεύθυνσης (IN), ώστε να αποφεύγονται λάθη χρήστη και να τηρούνται οι επιχειρησιακοί κανόνες της εφαρμογής.



Εικόνα 4: Σελίδα Έργα της εφαρμογής.

Στην ενότητα **Εκθέσεις** υλοποιήσαμε μία ολοκληρωμένη σελίδα διαχείρισης, με στόχο να καλύπτει τόσο τον προγραμματισμό όσο και την παρακολούθηση μιας έκθεσης (διάρκεια, επιμελητής, έργα που συμμετέχουν και κρατήσεις). Για πιο καθαρή οργάνωση χωρίσαμε τις εκθέσεις σε δύο tabs: Επερχόμενες και Παλαιές, όπου ο διαχωρισμός γίνεται με βάση την ημερομηνία λήξης (αν δεν υπάρχει λήξη, θεωρείται ίδια με την έναρξη). Τα δεδομένα εμφανίζονται σε πίνακες με βασικά πεδία (τίτλος, περίοδος, επιμελητής, μέγιστες/κρατημένες θέσεις, αριθμός έργων και σύντομη περιγραφή), ώστε ο admin να έχει γρήγορη εικόνα. Παράλληλα προσθέσαμε δυνατότητα προβολής λεπτομερειών για την επιλεγμένη έκθεση (dialog), όπου εμφανίζονται και τα έργα που συμμετέχουν.

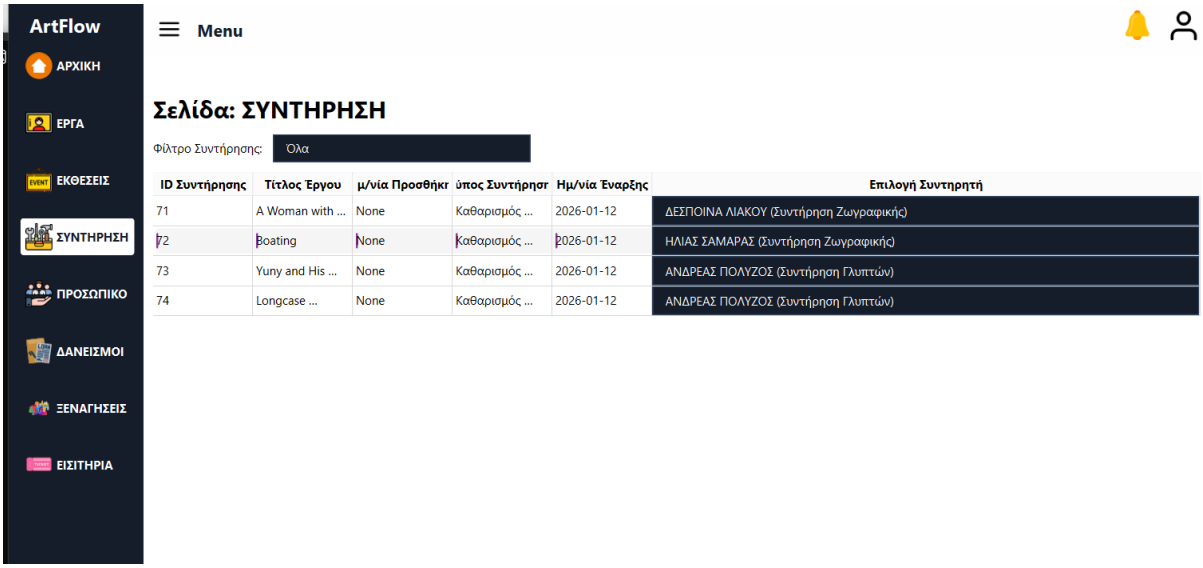
Για τη δημιουργία/επεξεργασία χρησιμοποιήσαμε ExhibitionDialog με φόρμα (τίτλος, ημερομηνίες έναρξης–λήξης, επιμελητής από λίστα προσωπικού, μέγιστες θέσεις και περιγραφή σε rich text). Η επιλογή των έργων δεν γίνεται με απλή εισαγωγή ids, αλλά μέσω ξεχωριστού works picker dialog, όπου ο χρήστης επιλέγει έργα με checkbox και μπορεί να ορίσει και σειρά εμφάνισης και σημείωση για κάθε έργο. Κατά την αποθήκευση εφαρμόζονται βασικοί έλεγχοι εγκυρότητας (υποχρεωτικός τίτλος, λήξη ≥ έναρξη και τουλάχιστον 1 επιλεγμένο έργο) και η καταχώρηση υλοποιείται με εισαγωγή/ενημέρωση της έκθεσης και ενημέρωση της συσχέτισης έκθεσης–έργων μέσω του ενδιάμεσου πίνακα. Επιπλέον, υποστηρίξαμε και το επιχειρησιακό κομμάτι χωρητικότητας: αν έχει οριστεί max_seats, το σύστημα εισιτηρίων ελέγχει τις ήδη ενεργές κρατήσεις και δεν επιτρέπει να ξεπεραστεί η διαθέσιμη χωρητικότητα. Τέλος, μετά από κάθε αλλαγή (add/edit/delete) γίνεται ενιαίο refresh ώστε να ενημερώνονται οι πίνακες εκθέσεων, το ημερολόγιο/ειδοποιήσεις και ο μετρητής της Αρχικής.



Εικόνα 5: Σελίδα Εκθέσεις της εφαρμογής.

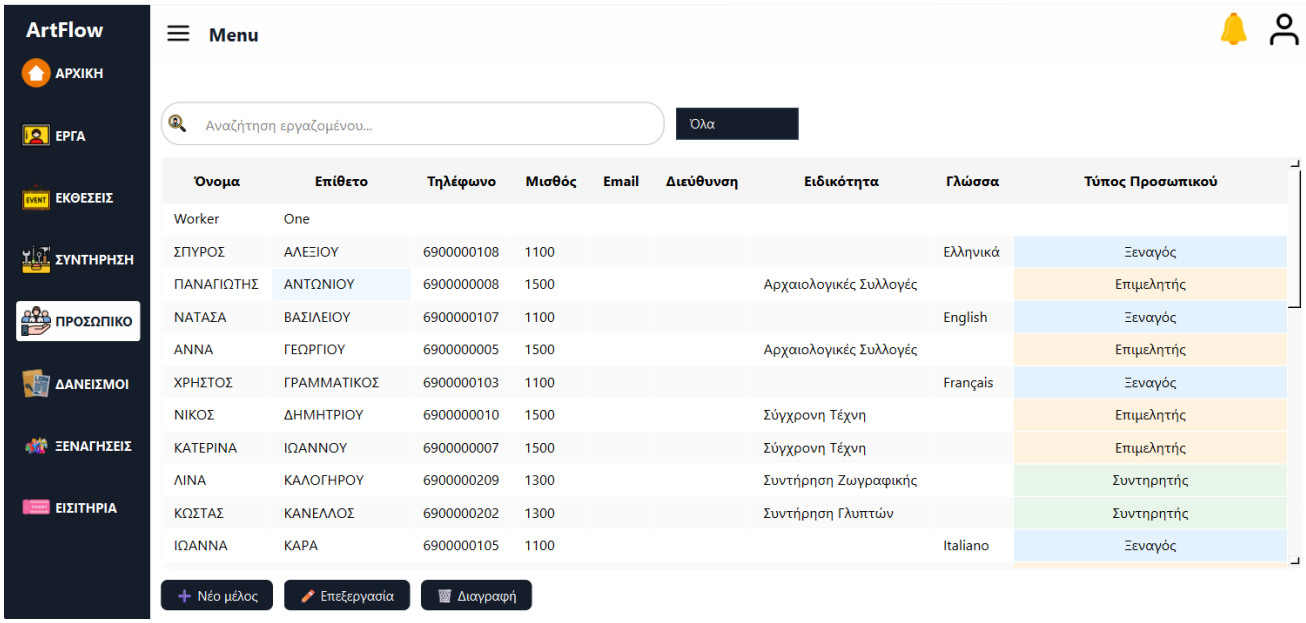
Στην ενότητα **Συντήρηση** υλοποιήσαμε μια σελίδα παρακολούθησης και οργάνωσης των εργασιών συντήρησης, με στόχο ο admin να βλέπει συγκεντρωμένα όλες τις ενεργές (ανοιχτές) συντηρήσεις και να μπορεί εύκολα να τις φιλτράρει και να αναθέτει υπεύθυνο. Η σελίδα σχεδιάστηκε με απλή δομή: τίτλος, φίλτρο τύπου συντήρησης και ένας πίνακας (QTableWidget) που εμφανίζει βασικές πληροφορίες για κάθε εγγραφή. Συγκεκριμένα, φορτώνουμε μόνο τις εγγραφές που είναι “ανοιχτές” (δηλαδή end IS NULL) και τις ταξινομούμε χρονολογικά με βάση την ημερομηνία έναρξης, ώστε να φαίνεται προτεραιοποίηση/εκκρεμότητα. Το φίλτρο επιτρέπει προβολή είτε Όλων είτε ανά συγκεκριμένο τύπο συντήρησης (π.χ. καθαρισμός καμβά ή γλυπτού), και κάθε αλλαγή φίλτρου προκαλεί ανανέωση του πίνακα.

Για την ανάθεση συντηρητή, επιλέξαμε να μην ανοίγουμε ξεχωριστό παράθυρο ανά γραμμή, αλλά να ενσωματώσουμε drop-down επιλογή μέσα στον πίνακα. Έτσι, στην τελευταία στήλη κάθε εγγραφής τοποθετούμε QComboBox που γεμίζει δυναμικά με τους διαθέσιμους συντηρητές από τη βάση (ένωση staff με conservators, ώστε να εμφανίζονται μόνο όσοι είναι πραγματικά συντηρητές μαζί με την ειδίκευσή τους). Αν μια συντήρηση έχει ήδη ανατεθεί, το combo προεπιλέγει τον αντίστοιχο συντηρητή, ενώ όταν ο χρήστης αλλάξει επιλογή, γίνεται άμεσα ενημέρωση στη βάση με UPDATE maintenance SET conservator_id = ..., ώστε η ανάθεση να αποθηκεύεται χωρίς επιπλέον βήματα. Με αυτόν τον τρόπο πετύχαμε μια γρήγορη και πρακτική ροή διαχείρισης συντηρήσεων με καθαρή παρουσίαση δεδομένων και απευθείας σύνδεση UI–βάσης.



Εικόνα 6: Σελίδα Συντήρηση της εφαρμογής.

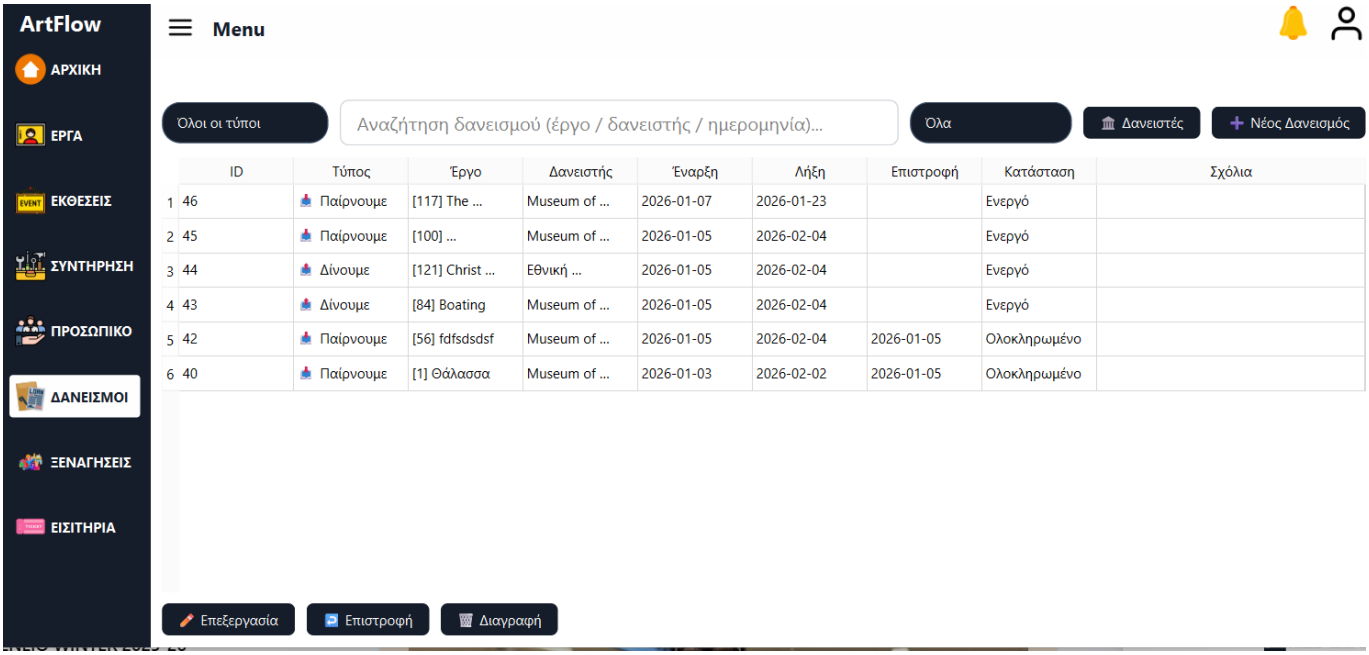
Στην ενότητα **Προσωπικό** υλοποιήσαμε μια σελίδα διαχείρισης των εργαζομένων του μουσείου, με στόχο να μπορούμε να καταχωρούμε και να ενημερώνουμε τόσο τα γενικά στοιχεία ενός μέλους προσωπικού (όνομα, τηλέφωνο, email), όσο και τα ρόλο-ειδικά χαρακτηριστικά ανά κατηγορία (ξεναγός/συντηρητής/επιμελητής). Για αυτό το λόγο οργανώσαμε τη σελίδα με tabs ανά ρόλο (Guides, Conservators, Curators) και έναν πίνακα ανά tab, ώστε ο admin να βλέπει κάθε φορά μόνο τα σχετικά πεδία και να μην υπάρχει “θόρυβος” από άσχετες πληροφορίες.Η λογική στη βάση είναι ότι τα κοινά στοιχεία βρίσκονται σε έναν κεντρικό πίνακα staff, ενώ τα ρόλο-ειδικά δεδομένα κρατιούνται σε επιμέρους πίνακες (π.χ. guides, conservators, curators) με σύνδεση μέσω ξένου κλειδιού στο staff_id. Έτσι, όταν προσθέτουμε νέο άτομο, ακολουθούμε βήματα αντίστοιχα με: πρώτα κάνουμε INSERT INTO staff (...) VALUES (...) και στη συνέχεια, ανάλογα με τον ρόλο, κάνουμε δεύτερη εισαγωγή στον αντίστοιχο πίνακα, π.χ. INSERT INTO guides(staff_id, languages, hourly_rate) VALUES (...). Αυτό μας επιτρέπει να κρατάμε τα δεδομένα κανονικοποιημένα και να αποφεύγουμε άδεια/άσχετα πεδία σε έναν ενιαίο πίνακα.Στο επίπεδο του UI, για εισαγωγή/επεξεργασία χρησιμοποιήσαμε αντίστοιχους διαλόγους (staff dialogs) ώστε να εφαρμόζονται έλεγχοι εγκυρότητας (π.χ. υποχρεωτικό όνομα, σωστό email, αριθμητικά πεδία μισθού/αμοιβής όπου υπάρχουν). Με την αποθήκευση, η εφαρμογή καλεί τις συναρτήσεις του db layer και εκτελεί τα κατάλληλα INSERT/UPDATE είτε στον βασικό πίνακα είτε στον ρόλο-πίνακα. Για παράδειγμα, σε αλλαγή στοιχείων επικοινωνίας γίνεται ενημέρωση τύπου UPDATE staff SET phone = ?, email = ? WHERE id = ?, ενώ σε αλλαγή χαρακτηριστικών ρόλου (π.χ. ειδίκευση συντηρητή) γίνεται UPDATE conservators SET specialty = ? WHERE staff_id = ?. Μετά από κάθε προσθήκη/επεξεργασία/διαγραφή, γίνεται refresh στο αντίστοιχο tab ώστε να ενημερώνεται ο πίνακας, αλλά και τα counters της Αρχικής (π.χ. συνολικό προσωπικό και κατανομή ανά ρόλο).Τέλος, δώσαμε έμφαση στη διαχείριση διαγραφών με ασφάλεια: πριν αφαιρέσουμε ένα άτομο, ελέγχουμε αν συμμετέχει σε ενεργές διαδικασίες (π.χ. προγραμματισμένες ξεναγήσεις, συντηρήσεις που δεν έχουν κλείσει) ώστε να μην δημιουργούνται ασυνέπειες. Όπου επιτρέπεται η διαγραφή, εφαρμόζουμε διαδοχική αφαίρεση από ρόλο-πίνακα και μετά από staff (ή αντίστοιχη ρύθμιση με foreign keys), ώστε να διατηρείται η ακεραιότητα της βάσης.



Εικόνα 7: Σελίδα Προσωπικό της εφαρμογής.

Στην ενότητα **Δανεισμοί** υλοποιήσαμε μια σελίδα που καλύπτει όλο τον κύκλο ζωής ενός δανεισμού (δημιουργία, ενημέρωση, επιστροφή, διαγραφή), με έμφαση στο να υπάρχει καθαρή εικόνα κατάστασης και να εφαρμόζονται οι επιχειρησιακοί κανόνες της εφαρμογής. Η σελίδα αποτελείται από top bar με πεδίο αναζήτησης και δύο φίλτρα, καθώς και έναν κεντρικό πίνακα (QTableWidget) που εμφανίζει όλες τις εγγραφές με πλήρη στοιχεία. Συγκεκριμένα, υλοποιήσαμε φίλτρο κατάστασης (Όλα / Ενεργά / Λήγουν σύντομα / Εκπρόθεσμα / Ολοκληρωμένα) και φίλτρο κατεύθυνσης (OUT: δανείζουμε εμείς / IN: δανειζόμαστε από τρίτους), ώστε ο admin να μπορεί να εντοπίζει άμεσα τις κρίσιμες περιπτώσεις (π.χ. εκπρόθεσμα ή κοντά στη λήξη). Η “Κατάσταση” δεν αποθηκεύεται ως ξεχωριστό πεδίο, αλλά υπολογίζεται από τις ημερομηνίες: αν υπάρχει return_date τότε θεωρείται ολοκληρωμένο, αλλιώς με βάση το end_date χαρακτηρίζεται ως “λήγει σύντομα” (π.χ. μέσα στις επόμενες 7 ημέρες) ή “εκπρόθεσμο” (αν η λήξη είναι στο παρελθόν). Παράλληλα, ο πίνακας εμφανίζει στήλες όπως ID, τύπος (IN/OUT), έργο, δανειστής, ημερομηνίες έναρξης/λήξης/επιστροφής, κατάσταση και σχόλια, με τα δεδομένα να έρχονται από join query (loans–works–borrowers) μέσω του db layer.

Για προσθήκη και επεξεργασία δανεισμού χρησιμοποιήσαμε τον LoanDialog, όπου ο χρήστης επιλέγει έργο, δανειστή, ημερομηνίες και κατεύθυνση (IN/OUT). Στη δημιουργία γίνεται καταχώρηση στη βάση με εντολή τύπου INSERT INTO loans (work_id, borrower_id, start_date, end_date, return_date, notes, direction) VALUES (..., NULL, ...), ενώ εφαρμόζεται και βασικός περιορισμός ώστε να μην επιτρέπεται δεύτερος ενεργός δανεισμός στο ίδιο έργο (έλεγχος όπως SELECT id FROM loans WHERE work_id=? AND return_date IS NULL και εμφάνιση μηνύματος αν υπάρχει ήδη ενεργός). Επιπλέον, όταν ο δανεισμός είναι τύπου IN (δηλαδή “δανειζόμαστε”), ενημερώνουμε το ίδιο το έργο ώστε να χαρακτηριστεί ως εξωτερικό, με ενημέρωση τύπουUPDATE works SET is_external=1 WHERE id=?,ώστε να φαίνεται σωστά και σε άλλες ενότητες (π.χ. Έργα / Dashboard).Η επιστροφή δανεισμού υλοποιείται με ξεκάθαρη ροή: με επιλογή γραμμής και επιβεβαίωση, ενημερώνουμε τον δανεισμό με UPDATE loans SET return_date=? WHERE id=?,ώστε να κλείσει επίσημα. Σε ειδική περίπτωση, αν ο δανεισμός ήταν IN (δηλαδή έργο που είχαμε προσωρινά), μετά την επιστροφή κάνουμε “αρχειοθέτηση” του έργου (soft-delete) ώστε να μη συνεχίζει να εμφανίζεται στη συλλογή, αλλά να διατηρείται το ιστορικό. Αυτό υλοποιείται με ενημέρωση τύπου UPDATE works SET is_archived=1, hall_id=NULL WHERE id=?, ενώ καθαρίζουμε και τυχόν σχετικές εγγραφές συντήρησης για να μην εμφανίζονται πλέον ως ενεργές. Τέλος, στη σελίδα προσθέσαμε και λειτουργία διαχείρισης δανειστών (Borrowers Manager) μέσω ξεχωριστού dialog, ώστε ο admin να μπορεί να κάνει CRUD στους φορείς/ιδρύματα που εμφανίζονται στη λίστα επιλογής των δανεισμών (π.χ. προσθήκη νέου δανειστή με INSERT INTO borrowers ..., επεξεργασία με UPDATE borrowers ..., διαγραφή με DELETE FROM borrowers ... όπου επιτρέπεται). Μετά από κάθε ενέργεια (add/edit/return/delete) γίνεται ανανέωση του πίνακα δανεισμών, καθώς και refresh σε σχετικές περιοχές της εφαρμογής (π.χ. ημερολόγιο/Αρχική και gallery έργων), ώστε το UI να παραμένει συνεπές με την πραγματική κατάσταση της βάσης.



Εικόνα 7: Σελίδα Δανεισμοί της εφαρμογής.

Στην ενότητα **Ξεναγήσεις** υλοποιήσαμε μια σελίδα προγραμματισμού και διαχείρισης ξεναγήσεων, με στόχο να καλύπτει τόσο τον προγραμματισμό συνεδριών (tour sessions) όσο και τις κρατήσεις επισκεπτών. Παράλληλα εφαρμόσαμε τον βασικό κανόνα ότι ο ίδιος ξεναγός δεν μπορεί να προγραμματίζεται σε επικαλυπτόμενα χρονικά διαστήματα. Η σελίδα οργανώθηκε με διαχειριστική λογική: περιλαμβάνει φίλτρα ημερομηνίας/κατάστασης και έναν κεντρικό πίνακα (QTableWidget) με όλες τις προγραμματισμένες ξεναγήσεις, όπου φαίνονται άμεσα τα κρίσιμα στοιχεία (ημερομηνία–ώρα, διάρκεια, γλώσσα, ξεναγός, μέγιστες θέσεις, κρατήσεις, κατάσταση και σημειώσεις). Έτσι ο admin μπορεί να εντοπίζει εύκολα ποια sessions είναι γεμάτα, ποια πλησιάζουν χρονικά και ποια έχουν ακυρωθεί.Για τη δημιουργία και επεξεργασία ξεναγήσεων χρησιμοποιήσαμε dialog (TourSessionDialog), στο οποίο ο χρήστης ορίζει ημερομηνία/ώρα έναρξης, διάρκεια (ή ώρα λήξης), γλώσσα, τύπο ξενάγησης, ξεναγό από λίστα και μέγιστες θέσεις. Κατά την αποθήκευση γίνεται εισαγωγή στη βάση με εντολή τύπου:

INSERT INTO tour_sessions (guide_id, start_dt, end_dt, language, type, max_seats, status, notes) VALUES (...)

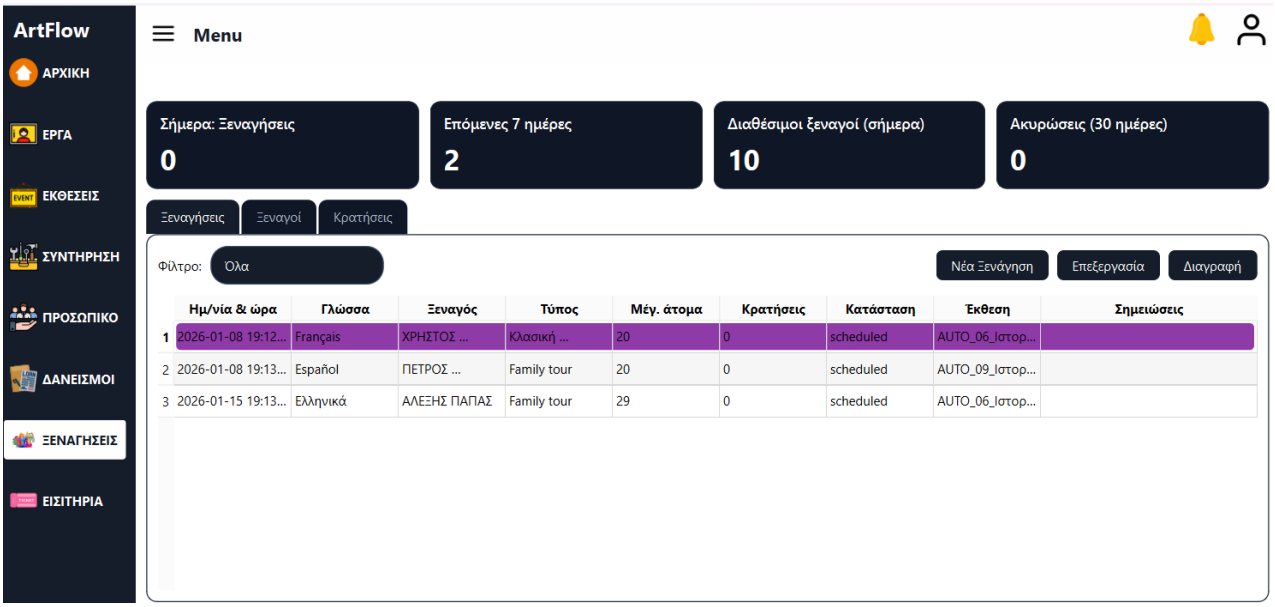
ή αντίστοιχα ενημέρωση με: UPDATE tour_sessions SET ... WHERE id = ?

Το σημαντικό σημείο είναι ότι πριν επιτρέψουμε την καταχώρηση, ελέγχουμε για επικάλυψη στο πρόγραμμα του ξεναγού. Εκτελούμε query που αναζητά αν υπάρχει άλλο session για τον ίδιο guide_id όπου τα χρονικά διαστήματα τέμνονται (π.χ. NOT (new_end <= start_dt OR new_start >= end_dt)). Αν βρεθεί αποτέλεσμα, εμφανίζουμε μήνυμα και μπλοκάρουμε την αποθήκευση, ώστε να αποφεύγονται διπλοκρατήσειςΠαράλληλα, για τις κρατήσεις υλοποιήσαμε ξεχωριστό dialog (TourBookingDialog) που συνδέεται με επιλεγμένο session και επιτρέπει καταχώρηση ονόματος/στοιχείων επισκέπτη και αριθμού ατόμων. Η καταχώρηση γίνεται στον πίνακα bookings με εντολή τύπου:

INSERT INTO tour_bookings (session_id, name, people_count, created_at) VALUES (...)

Πριν ολοκληρωθεί η κράτηση γίνεται έλεγχος χωρητικότητας: υπολογίζουμε πόσα άτομα έχουν ήδη κλείσει για το session (π.χ. SELECT SUM(people_count) FROM tour_bookings WHERE session_id=? AND status!='canceled') και δεν επιτρέπουμε να ξεπεραστεί το max_seats. Υποστηρίζουμε επίσης ακύρωση κράτησης με ενημέρωση τύπου:UPDATE tour_bookings SET status='canceled' WHERE id=? ,ώστε να διατηρείται ιστορικό κρατήσεων χωρίς να χάνεται η πληροφορία.Τέλος, προσθέσαμε λειτουργίες κατάστασης σε επίπεδο session (π.χ. active/canceled), ώστε ο admin να μπορεί να ακυρώνει μια ξενάγηση με εντολή τύπου:UPDATE tour_sessions SET status='canceled' WHERE id=? και το UI να ενημερώνεται άμεσα (π.χ. εμφάνιση “Canceled” και απενεργοποίηση ενεργειών κράτησης). Η ενότητα συνδέθηκε και με το Ημερολόγιο της Αρχικής, όπου τα tour sessions εμφανίζονται ως γεγονότα με διαφορετική

χρωματική σήμανση, ώστε να φαίνεται ο συνολικός προγραμματισμός του μουσείου σε ημερήσια βάση.



Εικόνα 8: Σελίδα Ξεναγήσεις της εφαρμογής.

Στην ενότητα **Εισιτήρια** υλοποιήσαμε ένα ολοκληρωμένο υποσύστημα έκδοσης και ελέγχου εισόδου, με στόχο να υποστηρίζει τόσο την πώληση/έκδοση εισιτηρίων όσο και τον έλεγχο εγκυρότητας κατά την είσοδο, διατηρώντας παράλληλα πλήρες ιστορικό για ιχνηλασιμότητα. Η σελίδα οργανώθηκε σε δύο βασικά κομμάτια: (α) λειτουργίες έκδοσης/διαχείρισης εισιτηρίων και (β) λειτουργίες scan (έλεγχος εισόδου). Τα δεδομένα εμφανίζονται σε πίνακα (QTableWidget) με πεδία όπως κωδικός εισιτηρίου, τύπος, τιμή, ημερομηνία έκδοσης, κατάσταση (*valid/used/canceled*) και, όπου υπάρχει, συσχέτιση με έκθεση/ξενάγηση. Η κεντρική λογική του συστήματος είναι ότι κάθε εισιτήριο έχει μοναδικό κωδικό και μία σαφή κατάσταση ζωής, ώστε να αποτρέπεται η διπλή χρήση ή η χρήση ακυρωμένου εισιτηρίου.

Επιπλέον, στο σύστημα εισιτηρίων δεν αποθηκεύουμε μόνο “μεμονωμένα εισιτήρια”, αλλά και τη λογική της συναλλαγής/παραγγελίας. Για αυτό χρησιμοποιήσαμε δύο πίνακες τύπου *order header – order lines*: τον πίνακα order_tickets και τον πίνακα order_ticket_items. Η ιδέα είναι ότι μια αγορά/έκδοση μπορεί να περιλαμβάνει πολλά εισιτήρια μαζί (π.χ. 2 κανονικά + 1 μειωμένο), άρα χρειαζόμαστε ένα “κεφάλι” παραγγελίας που κρατάει τα γενικά στοιχεία (ημερομηνία, σύνολο κ.λπ.) και ξεχωριστές “γραμμές” που περιγράφουν αναλυτικά τι περιλαμβάνει. Ο πίνακας order_tickets αποθηκεύει τα συνολικά στοιχεία μιας συναλλαγής (π.χ. created_at, total_amount), ενώ ο πίνακας order_ticket_items αποθηκεύει αναλυτικά τις γραμμές ανά τύπο εισιτηρίου (π.χ. ποσότητα, τιμή μονάδας) συνδεδεμένες με order_id. Με αυτό τον σχεδιασμό υποστηρίζουμε φυσιολογικά ομαδική αγορά, κρατάμε καθαρό ιστορικό συναλλαγών και μπορούμε να παράγουμε σωστά στατιστικά εσόδων και πωλήσεων.

Κατά την έκδοση εισιτηρίου, ο admin επιλέγει τύπο (π.χ. γενικό, μειωμένο) και όπου εφαρμόζεται μπορεί να συνδεθεί με συγκεκριμένη έκθεση/εκδήλωση. Η εισαγωγή στη βάση γίνεται με εντολή τύπου:

INSERT INTO tickets (code, type, price, issued_at, status, exhibition_id) VALUES (..., 'valid', ...)

Ο κωδικός (code) δημιουργείται έτσι ώστε να είναι μοναδικός (π.χ. random/UUID λογική) και πριν ολοκληρωθεί η εισαγωγή γίνεται έλεγχος ώστε να μη υπάρχει ήδη ίδιος κωδικός στη βάση. Επιπλέον, όταν υπάρχει περιορισμός χωρητικότητας (π.χ. έκθεση με μέγιστες θέσεις), ελέγχουμε τη διαθέσιμη χωρητικότητα πριν επιτραπεί η έκδοση, ώστε να μη δημιουργούνται υπεράριθμες κρατήσεις.

Για τη διαχείριση κατάστασης εισιτηρίων υποστηρίζουμε ακύρωση μέσω ενημέρωσης στη βάση με εντολή τύπου:

UPDATE tickets SET status='canceled' WHERE id=?

Με αυτόν τον τρόπο διατηρείται ιστορικό και αποτρέπεται η χρήση ακυρωμένου εισιτηρίου χωρίς να διαγράφονται δεδομένα. Αντίστοιχα, η χρήση (είσοδος) δεν γίνεται με απλή αλλαγή της κατάστασης, αλλά περνάει από διαδικασία ελέγχου ώστε να καταγράφεται και το γεγονός του scan.

Στο κομμάτι ελέγχου εισόδου (scan), ο χρήστης εισάγει/σκανάρει τον κωδικό εισιτηρίου και η εφαρμογή εκτελεί αναζήτηση τύπου:

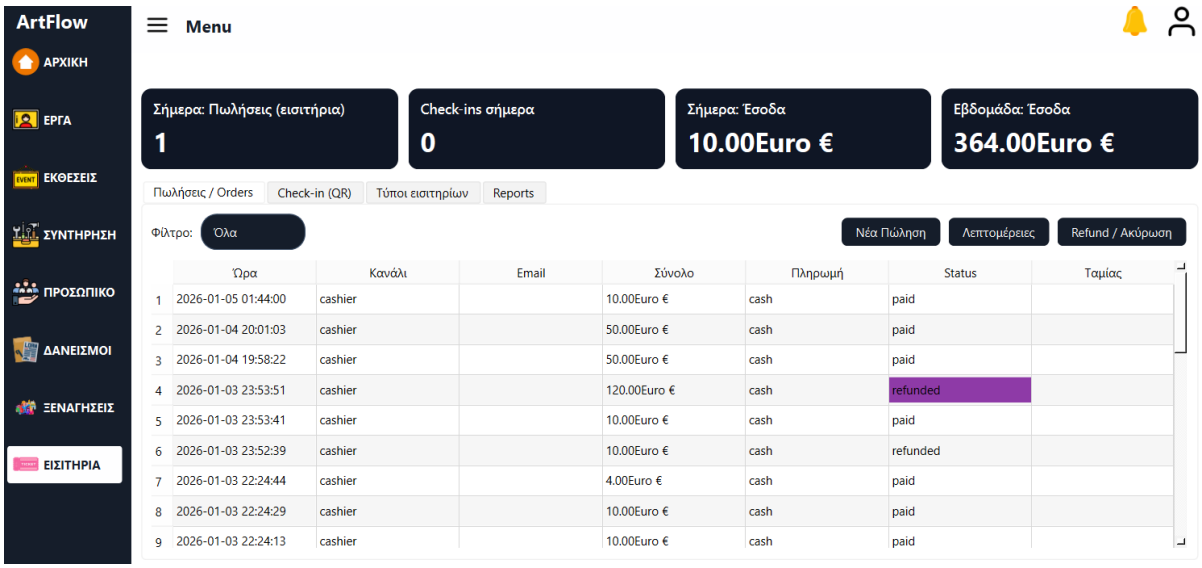
SELECT id, status FROM tickets WHERE code=?

Αν το εισιτήριο δεν βρεθεί, εμφανίζεται μήνυμα “invalid”. Αν βρεθεί αλλά η κατάσταση είναι canceled ή used, απορρίπτεται με αντίστοιχο μήνυμα. Αν είναι valid, τότε ολοκληρώνεται ο έλεγχος με δύο ενέργειες στη βάση:

- αλλαγή κατάστασης σε used: UPDATE tickets SET status='used' WHERE id=?
- καταγραφή του ελέγχου σε log: INSERT INTO ticket_scans (ticket_id, scanned_at, result) VALUES (?, CURRENT_TIMESTAMP, 'accepted')

(ή αντίστοιχα result='rejected' για αποτυχημένες προσπάθειες). Έτσι επιτυγχάνεται πλήρης ιχνηλασιμότητα, αφού μπορούμε να γνωρίζουμε πότε και πόσες φορές επιχειρήθηκε έλεγχος για κάθε εισιτήριο, ακόμη και σε περιπτώσεις απόρριψης.

Τέλος, ενσωματώσαμε λειτουργίες αναζήτησης/φίλτρων στον πίνακα εισιτηρίων (π.χ. ανά κατάσταση ή ανά χρονικό διάστημα), ώστε ο admin να βλέπει εύκολα πωλήσεις, ακυρώσεις και χρησιμοποιημένα εισιτήρια. Επιπλέον, τα δεδομένα από τα εισιτήρια τροφοδοτούν και το γράφημα στατιστικών της Αρχικής, όπου εμφανίζεται το έσοδο ανά ημέρα/μήνα ή ανά τύπο, μέσω συγκεντρωτικών queries από το db layer. Με αυτόν τον σχεδιασμό εξασφαλίσαμε ότι το σύστημα εισιτηρίων είναι απλό στη χρήση, ασφαλές ως προς την εγκυρότητα και πλήρες ως προς την καταγραφή και τον έλεγχο.



Εικόνα 9: Σελίδα Εισιτήρια της εφαρμογής.

Ολοκληρώνοντας, ελέγξαμε την ορθότητα της υλοποίησης με εισαγωγή δοκιμαστικών δεδομένων και εκτέλεση ολοκληρωμένων σεναρίων χρήσης: δημιουργία καλλιτεχνών/έργων, δημιουργία εκθέσεων και αντιστοίχιση έργων, καταγραφή συντηρήσεων, δανεισμοί και επιστροφές με τους αντίστοιχους περιορισμούς, προγραμματισμός ξεναγήσεων

και κρατήσεων χωρίς επικαλύψεις, καθώς και έκδοση και έλεγχος εισιτηρίων με καταγραφή των ελέγχων. Με αυτόν τον τρόπο επιβεβαιώσαμε ότι η βάση δεδομένων, οι περιορισμοί ακεραιότητας και το admin dashboard συνεργάζονται σωστά και καλύπτουν τον στόχο της εργασίας.

3 ΑΞΙΟΛΟΓΗΣΗ

Η αξιολόγηση της εργασίας μας έγινε σταδιακά και ακολούθησε τα ίδια στάδια με την ανάπτυξη του συστήματος, ώστε να εντοπίζουμε προβλήματα νωρίς και να αποφεύγουμε μεγάλες αλλαγές στο τέλος. Αρχικά, αξιολογήσαμε το εννοιολογικό μοντέλο ελέγχοντας αν ο μικρόκοσμος που περιγράφουμε καλύπτει πράγματι τις ανάγκες μιας πινακοθήκης και αν οι οντότητες/συσχετίσεις που επιλέξαμε οδηγούν σε ρεαλιστικά σενάρια χρήσης. Σε αυτό το σημείο εστιάσαμε κυρίως στη σωστή επιλογή πληθικοτήτων, ώστε να μη λείπουν κρίσιμες συσχετίσεις (π.χ. έκθεση–έργα, ξεναγήσεις–κρατήσεις, δανεισμοί–έργα) αλλά και να αποφεύγονται υπερβολικές οντότητες που δεν προσφέρουν λειτουργικότητα. Η ύπαρξη του ERD μάς βοήθησε να εντοπίσουμε ασυνέπειες στη λογική πριν περάσουμε στην υλοποίηση.

Στη συνέχεια, κατά τον μετασχηματισμό σε σχεσιακό σχήμα, η αξιολόγηση έγινε με βάση το αν το τελικό μοντέλο αποτυπώνει σωστά τις σχέσεις του ERD και αν επιτυγχάνεται ο σωστός βαθμός κανονικοποίησης. Ελέγξαμε ειδικά τις περιπτώσεις πολλών-προς-πολλούς και τα σημεία όπου απαιτούνται ενδιάμεσοι πίνακες, καθώς και το αν τα ξένα κλειδιά “δένουν” σωστά τους πίνακες. Παράλληλα αξιολογήσαμε αν το σχήμα επιτρέπει ιστορικό όπου χρειάζεται (π.χ. συντηρήσεις, δανεισμοί, scan logs), ώστε να μην χάνεται πληροφορία με την πάροδο του χρόνου.

Για την αξιολόγηση της υλοποίησης της βάσης σε SQLite, βασικός μας στόχος ήταν να επιβεβαιώσουμε ότι εφαρμόζονται στην πράξη οι κανόνες ακεραιότητας και οι επιχειρησιακοί περιορισμοί που είχαμε θέσει. Ελέγξαμε ότι τα πρωτεύοντα/ξένα κλειδιά λειτουργούν σωστά, ότι οι μοναδικότητες αποτρέπουν διπλοεγγραφές και ότι ειδικοί κανόνες, όπως ο περιορισμός “ένας ενεργός δανεισμός ανά έργο”, τηρούνται από το σύστημα. Επίσης δοκιμάσαμε τις ροές εισιτηρίων (μοναδικοί κωδικοί, καταστάσεις valid/used/canceled, έλεγχος ημερομηνίας) και επιβεβαιώσαμε ότι κάθε έλεγχος εισόδου καταγράφεται στο scan log, ώστε να υπάρχει πλήρης ιχνηλασιμότητα.

Τέλος, η εφαρμογή αξιολογήθηκε με κριτήριο το κατά πόσο προσφέρει ένα ολοκληρωμένο και σταθερό περιβάλλον διαχείρισης. Προτεραιότητά μας ήταν η λειτουργικότητα: να μπορούν να εκτελεστούν από το dashboard όλες οι βασικές ενέργειες (CRUD και βασικές αναζητήσεις) και, κυρίως, να υποστηρίζονται σωστά πιο σύνθετα σενάρια όπως δανεισμός–επιστροφή, προγραμματισμός ξεναγήσεων χωρίς χρονικές επικαλύψεις, έκδοση εισιτηρίων και check-in με καταγραφή. Για να το πετύχουμε, δοκιμάσαμε συστηματικά την εφαρμογή με εισαγωγή δεδομένων και επαναλαμβανόμενες δοκιμές σε ακραίες περιπτώσεις (π.χ. προσπάθεια δεύτερου ενεργού δανεισμού, check-in ήδη χρησιμοποιημένου εισιτηρίου, επικαλυπτόμενες ώρες ξενάγησης). Έτσι καταλήξαμε σε μια εφαρμογή που δίνει έμφαση στη σωστή λειτουργία και στη συνέπεια των δεδομένων, με διεπαφή αρκετά καθαρή ώστε να χρησιμοποιείται εύκολα από τον διαχειριστή.

4 ΔΕΔΟΜΕΝΑ

Για να μπορέσουμε να υλοποιήσουμε και να αξιολογήσουμε σωστά την εφαρμογή, χρειαζόμασταν ένα ρεαλιστικό σύνολο δεδομένων μέσα στη βάση. Δηλαδή, έπρεπε να υπάρχουν από την αρχή αρκετά έργα/καλλιτέχνες ώστε να μπορούμε να κάνουμε δοκιμές (αναζητήσεις, φίλτρα, προβολές, συσχετίσεις), αλλά και δεδομένα που προκύπτουν από την καθημερινή λειτουργία (δανεισμοί, εισιτήρια, εκθέσεις κλπ.). Πηγές δεδομένων (έργα τέχνης): Για τα δεδομένα των έργων χρησιμοποιήσαμε δύο αξιόπιστες και έτοιμες πηγές που διαθέτουν δημόσια APIs: The Metropolitan Museum of Art (MET) και Art Institute of Chicago. Τις επιλέξαμε γιατί έχουν μεγάλο αριθμό καταγεγραμμένων έργων και αρκετά μεταδεδομένα (τίτλος, καλλιτέχνης, χρονολογία, τεχνική/υλικά, διαστάσεις κ.ά.), κάτι που μας έδωσε πιο “πραγματικά” δεδομένα σε σχέση με το να τα φτιάχναμε όλα χειροκίνητα. Έτσι, μπορέσαμε να γεμίσουμε τη βάση γρήγορα και με συνέπεια. Πώς τα ενσωματώσαμε στη βάση. Για την αρχική εισαγωγή των δεδομένων χρησιμοποιήσαμε το smart bulk import script. Με αυτό

το script κάναμε μαζική εισαγωγή των δεδομένων στη βάση μας, ώστε να μην χρειαστεί να περάσουμε εκατοντάδες εγγραφές μία-μία, να μειώσουμε πιθανά λάθη από χειροκίνητη καταχώρηση και να έχουμε άμεσα έτοιμο dataset για δοκιμές της εφαρμογής. Ουσιαστικά, το script φρόντισε να "γεμίσει" τις βασικές οντότητες της βάσης (π.χ. έργα/καλλιτέχνες), επιταχύνοντας σημαντικά τη διαδικασία αρχικοποίησης της ΒΔ. Τα υπόλοιπα δεδομένα, που είναι πιο λειτουργικά και προκύπτουν από τη χρήση της εφαρμογής, τα καταχωρήσαμε χειροκίνητα μέσα από το UI. Συγκεκριμένα, οι εγγραφές για Δανεισμούς, Εισιτήρια, Εκθέσεις, Ξεναγήσεις και Προσωπικό μπήκαν από τις αντίστοιχες φόρμες/οθόνες της εφαρμογής, ώστε να προσομοιώνεται η πραγματική λειτουργία του συστήματος.

Συνολικά, ο συνδυασμός μαζικής εισαγωγής με το smart bulk script (για γρήγορο και ρεαλιστικό αρχικό dataset από MET και Art Institute of Chicago) και χειροκίνητης εισαγωγής από την εφαρμογή (για δεδομένα καθημερινής χρήσης) μας βοήθησε να ολοκληρώσουμε πιο εύκολα την εισαγωγή δεδομένων και να κάνουμε πιο σωστές δοκιμές/αξιολόγηση του project.

5 ΚΥΡΙΕΣ ΕΝΕΡΓΕΙΕΣ - ΚΑΤΑΜΕΤΡΙΣΜΟΣ

Για να ολοκληρώσουμε το πρότζεκτ κάναμε αρχικά ανάλυση απαιτήσεων (τι λειτουργίες χρειάζεται ένα σύστημα διαχείρησης πινακοθήκης) και καταλήξαμε στις βασικές οντότητες/ροές (έργα, καλλιτέχνες, αίθουσες/τοποθετήσεις, εκθέσεις, δανεισμοί, ξεναγήσεις, εισιτήρια, προσωπικό, συντήρηση). Στη συνέχεια δουλέψαμε συνολικά σαν ομάδα στο ERD και στο στήσιμο του σχήματος της βάσης (πίνακες, σχέσεις, κλειδιά), ώστε να υπάρχει σωστή δομή και να "κουμπώνει" με τις ανάγκες της εφαρμογής. Μετά περάσαμε στην υλοποίηση: δημιουργήσαμε την εφαρμογή σε PySide6 με πολυσέλιδη πλοήγηση (home και ξεχωριστές ενότητες), φτιάξαμε τα dialogs/φόρμες για προσθήκη–επεξεργασία–διαγραφή εγγραφών και συνδέσαμε τις λειτουργίες με τη βάση (CRUD, έλεγχοι, ενημερώσεις). Παράλληλα ενσωματώσαμε δεδομένα: χρησιμοποιήσαμε smart bulk import script για γρήγορο αρχικό γέμισμα της βάσης (έργα/καλλιτέχνες) και μετά περάσαμε χειροκίνητα από την εφαρμογή τα λειτουργικά δεδομένα (π.χ. δανεισμούς, εισιτήρια, ξεναγήσεις κλπ.) για να δοκιμάζουμε ρεαλιστικά σενάρια. Τέλος κάναμε δοκιμές, διορθώσεις και "δέσιμο" των σελίδων ώστε η εφαρμογή να λειτουργεί σωστά από άκρη σε άκρη. Σε αδρές γραμμές, ο καταμερισμός ήταν: και οι δύο κάναμε ομαδικά το ERD, το schema και τις βασικές αποφάσεις αρχιτεκτονικής, ο Κωνσταντίνος ανέλαβε κυρίως τις σελίδες Εκθέσεις, Συντήρηση και Προσωπικό, ενώ εγώ ανέλαβα κυρίως τις ενότητες Έργα/Καλλιτέχνες, Δανεισμοί, Ξεναγήσεις και Εισιτήρια. Την αρχική σελίδα (home) και το τελικό έλεγχο ροών τα δουλέψαμε μαζί.

6 ΧΡΟΝΟΔΙΑΓΡΑΜΜΑ

Για να ολοκληρώσουμε το πρότζεκτ οργανώσαμε τη δουλειά μας χρονικά, ώστε πρώτα να "κλείσει" η βάση και οι βασικές οθόνες και μετά να γεμίσει η βάση δεδομένων με δεδομένα. Συνοπτικά το χρονοδιάγραμμα που ακολουθήσαμε ήταν:

Νοέμβριος: Συλλογή απαιτήσεων/ιδεών για το τι πρέπει να υποστηρίζει ένα σύστημα μουσείου και ομαδική δουλειά στο ERD και στο αρχικό schema (οντότητες, σχέσεις, κλειδιά).

Τέλη Νοεμβρίου – μέσα Δεκεμβρίου: Οριστικοποίηση του σχήματος και υλοποίηση της βάσης σε SQLite (πίνακες, περιορισμοί, σχέσεις) + πρώτη σύνδεση της εφαρμογής με τη ΒΔ. Παράλληλα ξεκινήσαμε το UI και την πλοήγηση/αρχική σελίδα.

Μέσα Δεκεμβρίου – τέλη Δεκεμβρίου: Κύρια ανάπτυξη της εφαρμογής σε Python (PySide6) και υλοποίηση των ενοτήτων.

Αρχές Ιανουαρίου: Τελικές διορθώσεις, έλεγχοι λειτουργιών (CRUD/φόρμες/validations) και στο τέλος εισαγωγή δεδομένων στη βάση: αρχικά μαζική εισαγωγή έργων/καλλιτεχνών με το smart bulk import script και μετά χειροκίνητη καταχώρηση από την εφαρμογή για τα λειτουργικά δεδομένα (δανεισμοί, εισιτήρια, εκθέσεις, ξεναγήσεις, προσωπικό), ώστε να γίνουν και ρεαλιστικές δοκιμές.

7 ΠΑΡΑΡΤΗΜΑ

7.1 ΟΔΗΓΙΕΣ ΕΓΚΑΤΑΣΤΑΣΗΣ

1) Προαπαιτούμενα

- Εγκατεστημένο Git
- Εγκατεστημένο Python (προτείνεται 3.10+)

2) Κατέβασμα κώδικα από GitHub

Άνοιξε ένα Terminal / PowerShell και τρέξε:
git clone < https://github.com/spirosmpt/---ArtFlow.git >
cd <TO_REPO_FOLDER>
(Εναλλακτικά: Download ZIP από GitHub και αποσυμπίεση, μετά μπαίνεις στον φάκελο.)

3) Δημιουργία virtual environment (ΠΡΟΤΕΙΝΕΤΑΙ)

Windows (PowerShell):
python -m venv .venv
.\.venv\Scripts\Activate.ps1
Windows (cmd):
python -m venv .venv
.\.venv\Scripts\activate.bat
macOS / Linux:
python3 -m venv .venv
source .venv/bin/activate
Για να σιγουρευτείς ότι είσαι μέσα στο venv:
python --version
pip --version

4) Εγκατάσταση βιβλιοθηκών (pip install)

Πρώτα αναβάθμισε pip:
python -m pip install --upgrade pip
στο repo υπάρχει requirements.txt:
pip install -r requirements.txt

5) Πρώτη εκτέλεση

Τρέξε:
python main.py
Η main.py:
- Καλεί init_db() (φτιάχνει/αναβαθμίζει τη βάση και demo δεδομένα)
- Ανοίγει Login παράθυρο
- Μετά ανοίγει το MainWindow

6) Στοιχεία σύνδεσης (demo)

Υπάρχει demo χρήστης που μπαίνει αυτόματα στη βάση αν δεν υπάρχει:
Username: worker1
Password: 1234
(Αν γράψεις λάθος στοιχεία, θα σου βγάλει μήνυμα “Λάθος στοιχεία σύνδεσης”.)

7) Προαιρετικό: Ρύθμιση OpenAI (μόνο αν χρησιμοποιήσεις το AI feature)

Κάποια λειτουργία της εφαρμογής απαιτεί να υπάρχει το OPENAI_API_KEY στο περιβάλλον.
Αν δεν υπάρχει, θα πετάξει error όταν πας να χρησιμοποιήσεις τη συγκεκριμένη λειτουργία.
Ρύθμιση (Windows PowerShell – μόνο για τρέχον terminal):
$env:OPENAI_API_KEY="ΒΑΛΕ_ΕΔΩ_TO_KEY"
Ρύθμιση (Windows – μόνιμα, θέλει νέο terminal μετά):
setx OPENAI_API_KEY "ΒΑΛΕ_ΕΔΩ_TO_KEY"
Ρύθμιση (macOS / Linux – τρέχον terminal):
export OPENAI_API_KEY="ΒΑΛΕ_ΕΔΩ_TO_KEY"
Μετά ξανατρέχεις:
python main.py

7.2 ΒΙΒΛΙΟΓΡΑΦΙΑ

- The Qt Company. *Qt for Python (PySide6) Documentation (Qt for Python 6.x)*. (Τεκμηρίωση). Πρόσβαση: 5 Ιανουαρίου 2026. doc.qt.io
- The Qt Company. *PySide6.QtWidgets — Qt for Python*. Πρόσβαση: 5 Ιανουαρίου 2026. doc.qt.io
- The Qt Company. *Signals and Slots — Qt for Python Tutorial*. Πρόσβαση: 5 Ιανουαρίου 2026. doc.qt.io
- The Qt Company. *Signals & Slots (Qt Core)*. Πρόσβαση: 5 Ιανουαρίου 2026. doc.qt.io
- The Qt Company. *The Qt Resource System (Qt Core)*. Πρόσβαση: 5 Ιανουαρίου 2026. doc.qt.io
- The Qt Company. *QPropertyAnimation Class (Qt Core)*. Πρόσβαση: 5 Ιανουαρίου 2026. doc.qt.io
- Matplotlib Development Team. *Matplotlib Documentation (3.10.x)*. Πρόσβαση: 5 Ιανουαρίου 2026. matplotlib.org
- Matplotlib Development Team. *matplotlib.backends.backend_qtagg (QtAgg backend API)*. Πρόσβαση: 5 Ιανουαρίου 2026. matplotlib.org
- Matplotlib Development Team. *Embed in Qt — Embedding Matplotlib in Qt Applications*. Πρόσβαση: 5 Ιανουαρίου 2026. matplotlib.org
- OpenAI. *OpenAI API Reference*. Πρόσβαση: 5 Ιανουαρίου 2026. platform.openai.com
- OpenAI. *Developer Quickstart (OpenAI API)*. Πρόσβαση: 5 Ιανουαρίου 2026. platform.openai.com
- OpenAI. *openai-python (Official Python library) — GitHub repository/README*. Πρόσβαση: 5 Ιανουαρίου 2026. GitHub
- SQLite Project. *SQLite Documentation Index*. Πρόσβαση: 5 Ιανουαρίου 2026. sqlite.org
- SQLite Project. *SQL As Understood By SQLite (Language Reference)*. Πρόσβαση: 5 Ιανουαρίου 2026. sqlite.org
- Python Software Foundation. *sqlite3 — DB-API 2.0 interface for SQLite databases (Python docs)*. Πρόσβαση: 5 Ιανουαρίου 2026. Python documentation
- Python Software Foundation. *PEP 249 — Python Database API Specification v2.0*. Πρόσβαση: 5 Ιανουαρίου 2026. Python Enhancement Proposals (PEPs)
- Python Software Foundation. *hashlib — Secure hashes and message digests*. Πρόσβαση: 5 Ιανουαρίου 2026. Python documentation
- Python Software Foundation. *json — JSON encoder and decoder*. Πρόσβαση: 5 Ιανουαρίου 2026. Python documentation
- Python Software Foundation. *datetime — Basic date and time types*. Πρόσβαση: 5 Ιανουαρίου 2026. Python documentation
- Python Software Foundation. *re — Regular expression operations*. Πρόσβαση: 5 Ιανουαρίου 2026. Python documentation
- Python Software Foundation. *os — Miscellaneous operating system interfaces*. Πρόσβαση: 5 Ιανουαρίου 2026. Python documentation